Vision-Based Flow-Front Assessment in Vacuum-Assisted Resin Transfer Molding

J.C. Vaught[1], Marshall Pigford[2], Declan Johnson[3] and Darun Barazanchy[4]
University of South Carolina, Columbia, SC, 29208, USA

Vacuum-Assisted Resin Transfer Molding (VARTM) operators read the visible advance of the resin flow front under the transparent vacuum bag as their primary process indicator, yet the open toolchain for turning that video into a reproducible, time-resolved flow-front segmentation is thin. Published vision systems for VARTM and Liquid Composite Molding either ship as proprietary controllers tied to a specific cell or as deep convolutional networks trained on simulated frames whose distribution does not match a typical research-bench camera, leaving practitioners without a starting point. Here we show that a twelve-stage classical computer-vision pipeline implemented in Rust, configured via roughly fifty command-line flags, and shipped under the MIT license, segments resin flow fronts in VARTM video at thirty frames per second end-to-end on a single CPU. On a fifty-five-frame hand-labeled subset spanning eleven distinct VARTM runs, the default configuration achieves a mask Intersection-over-Union of $X.XXX$ (95 % bootstrap confidence interval $[a, b]$) and a boundary F_1 of $Y.YYY$ (CI $[c, d]$). The implementation is auditable end-to-end via bit-exact per-stage parity tests and exports annotations in COCO, YOLOv5, and Darknet formats, supplying the supervision needed to bootstrap a downstream learned detector and serving as a reproducible reference for permeability-inversion and process-monitoring studies that today have no shared baseline.

I. Nomenclature

F_t	=	converted-colorspace frame at time index t
G_t	=	reference image at time index t
$G_t^\star$	=	effective reference (peak map when peak-reference enabled, otherwise channel-zero of G_t)
P_t	=	per-pixel peak-brightness map at time t
D_t	=	per-pixel response field at time t
R	=	binary region-of-interest mask
H, W	=	frame height and width in pixels
C	=	channel count of the working colorspace
α	=	exponential-moving-average factor for running-reference mode
κ	=	sqrt-area growth factor estimated from calibration frames
ρ	=	target wet-fraction for dynamic reference mode
λ	=	user lag scale factor for dynamic reference
τ_t	=	elapsed time at frame index t in seconds
$\Delta\tau_t$	=	dynamic lag in seconds for the reference frame
w_c	=	channel weight in the delta computation
N_{calib}	=	number of calibration frames before dynamic reference activates

[1]Graduate Research Assistant, Department of Mechanical Engineering, AIAA Student Member.
[2]Undergraduate Research Assistant, Department of Computer Science and Engineering, AIAA Student Member.
[3]Graduate Research Assistant, Department of Mechanical Engineering, AIAA Student Member.
[4]Research Assistant Professor, Department of Mechanical Engineering, AIAA Member.

f = video frame rate in frames per second
IoU = Intersection-over-Union mask agreement score
F_1 = boundary F_1 score, mean across pixel tolerances $\tau \in \{1, 3, 5\}$

II. Introduction

Vacuum-Assisted Resin Transfer Molding (VARTM) is the workhorse process for one-off composite laminates from research panels through small-series aerospace structural parts, and the operator's chief lever during a run is the visible advance of the resin flow front under the transparent vacuum bag [1].
The position, shape, and rate of that front carry process-quality information that other, more instrumented modalities recover only indirectly. Distributed dielectric arrays [2] and area-sensor grids [3] produce excellent *local* wetness measurements but require sensors embedded in the lay-up, and fiber-optic frequency-domain reflectometry [4] demands instrumentation no operator on a research bench will install for a one-off panel. Direct video of the bag is the one modality that costs nothing extra, sees the entire laminate at once, and is already recorded in most labs as a procedural artifact.

Despite that, the open-source toolchain for turning that video into a usable, time-resolved flow-front segmentation is thin. The published vision systems for VARTM and Liquid Composite Molding (LCM) tend to ship as proprietary controllers tied to a specific cell [5,6], or as deep convolutional networks trained on simulated frames whose distribution does not match a typical research-bench camera [7]. The practitioner who wants to characterize a single infusion run today has no obvious starting point. Reimplementing the basic appearance-difference pipeline from primitives is feasible, but the design space is large enough — colorspaces, reference selection, threshold rules, morphology, temporal locking — that a clean, audited reference implementation would save the field substantial duplicated effort.

The work is hard for three concrete reasons that have to be addressed by anything claiming to do it. The vacuum bag produces strong specular reflections from lab lighting that look bright in every channel and shift on every operator move; the bag wrinkles and creases that pre-date the front are static dark features that do not represent wet fabric and must not be mistaken for it; and the lighting itself drifts across a multi-minute infusion as lamps warm up and the operator moves equipment. Any practical pipeline has to handle these without an embedded sensor, without a calibrated illumination rig, and without a per-run training set, because no research lab consistently has any of those.

This paper presents VRIFA, an open Rust-implemented classical-computer-vision pipeline that segments resin flow fronts in VARTM video at 30 frames per second end-to-end. On a 55-frame hand-labeled subset spanning eleven distinct VARTM runs, the default configuration achieves mask Intersection-over-Union (IoU) of $X.XXX$ with a 95 % bootstrap confidence interval of $[a, b]$, and a boundary F_1 of $Y.YYY$ with confidence interval $[c, d]$. The pipeline runs on commodity central-processing-unit (CPU) hardware without a graphics-processing unit (GPU), is auditable end-to-end with bit-exact per-stage parity tests, and exports annotations in Common Objects in Context (COCO), YOLOv5, and Darknet formats. The artifact is released under the Massachusetts Institute of Technology (MIT) license. The contribution is therefore Hybrid in the contribution-typology sense, with an Application core (classical computer vision retargeted to a domain that prior optical-input work treats only proprietarily) and a Systems wrapper (open implementation, three-tier benchmark, byte-exact reproducibility, and a downstream-detector-ready label format).

The contributions of this paper are the following:

1. a pipeline of twelve auditable stages that produces a binary flow-front mask, an overlay frame, a normalized contrast heatmap, and a Common Objects in Context annotation set from a single VARTM video, with no embedded sensors and no per-run training (Section 3);
2. a Rust workspace structured as five crates with a three-tier benchmark gate that isolates algorithm cost from encoder cost from export cost, plus an archived bit-exact stage-parity record over six diagnostic frames (Section 4);
3. a fifty-five-frame hand-labeled subset spanning eleven distinct VARTM runs, annotated under a documented protocol, on which the default configuration is reported with bootstrap-based confidence intervals at the overall level and broken down per sample (Sections 5 and 6);

4. a runtime characterization showing that the algorithm itself runs in roughly 23 seconds on a 706-frame, 1920 × 1080 infusion video, that is, near 30 frames per second on CPU, with the rest of the per-tier wall-clock attributable to encoder and per-frame export costs that are independent of the algorithm (Section 7); and
5. a documented protocol for bootstrapping a downstream detector from VRIFA's annotation export, demonstrated qualitatively on a YOLO model trained from the COCO output (Section 8).

The paper is organized as follows. Section 2 places VRIFA among the three families of prior work that touch the problem. Section 3 walks through the algorithm. Section 4 describes the implementation. Section 5 documents the labeling protocol and the eleven-sample dataset. Section 6 reports the agreement metrics. Section 7 characterizes runtime. Section 8 demonstrates detector bootstrap. Section 9 discusses failure modes and the honest tradeoffs the design makes. Section 10 concludes.

III. Related Work

Prior work on Vacuum-Assisted Resin Transfer Molding (VARTM) and the broader Liquid Composite Molding (LCM) family touches the flow front from three different directions. A first body of work *uses* flow-front information to act on the process through closed-loop control or fault response. A second body *observes* the process to characterize it, typically with embedded sensors or auxiliary cameras for offline study. A third body *models* the underlying Darcy physics or inverts it from sparse measurements. VRIFA sits at the seam of all three because it produces the per-frame full-field flow-front mask that the first family assumes is available, that the second family currently approximates with sensor grids, and that the third family typically receives only as a handful of digitized contours from a manual annotation pass. The remainder of this section walks each family in turn and closes with the specific white-space VRIFA fills.

A. Use: vision and sensors that act on the VARTM process

Closed-loop VARTM controllers have a long lineage. Multi-segment injection-line work coupled point sensors with a finite-element flow model so that valves could be re-sequenced in real time to chase the flow front toward starved regions and away from race-tracks [5]. The control loop is genuinely real time and the empirical results are convincing for plate geometries, but the feedback signal is a sparse vector of resin-arrival times at instrumented locations, which means dry-spot detection only works when a sensor happens to lie inside the dry region. VRIFA differs by treating every pixel inside the bag as a potential sensor, so dry-spot evidence is not gated on instrumentation density.

A more recent line of work replaces sensor pucks with a top-mounted camera. Mesogitis and co-workers built a webcam-plus-microcontroller rig that segments the resin flow front with classical machine-vision routines and toggles inlet valves on the resulting front position [8]. Mendizabal et al. close a similar loop on resin film infusion, regulating flow-front velocity to a fixed setpoint with a PID over an OpenCV pipeline and reporting tensile-strength gains of up to 18 percent at the optimal velocity window [6]. Both of these systems are valuable demonstrations that vision *alone* can drive VARTM control, but neither releases the segmentation pipeline as a runnable artifact, neither reports a labeled video benchmark, and both treat the segmentation step as a black box. VRIFA differs by publishing the pipeline source, the per-frame masks, and a labeled validation subset, so other groups can swap in alternative front detectors without rebuilding the apparatus.

Adjacent to control are vision-based defect-detection systems for composites more broadly [9]. These pipelines now lean almost entirely on convolutional or transformer detectors trained on hand-curated defect crops, and a recurring bottleneck across that survey is the cost of pixel-accurate labels. Snorkel-style weak supervision has reduced this cost in other domains by letting subject-matter experts encode heuristics as labeling functions and then learning a denoised label model [10]. VRIFA contributes a complementary lever, a deterministic classical-CV detector that produces per-pixel pseudo-labels at video rate, which can then seed a YOLO [11] or U-Net [12] trainer without a hand-segmentation campaign.

B. Observe: sensor-based and vision-based process characterization

A second family treats the flow front as something to be *measured* rather than acted on. Konstantopoulos and co-authors review the in-line sensing landscape and group the field around dielectric, thermal, ultrasonic, fiber-optic, and electrical-resistance sensors [1]. Dielectric tracking by Yenilmez and Sozer and others gives unsaturated and saturated front positions at each electrode pair, with an implementation footprint that is hard to beat for cost, but the

spatial resolution of the resulting front is limited by the electrode pitch [2]. Matsuzaki and co-workers dramatically improve coverage with an area-sensor array that tiles a polyimide film into a dense capacitive matrix and recovers the impregnated-area ratio per cell, which is the closest sensor-side analogue to a full-field flow-front mask [3]. The resulting front is, however, still a quantized grid that is destructive to instrument inside production tooling and that adds permeability to the stack-up.

Fiber-optic strategies use Fiber Bragg Grating arrays or Optical Frequency Domain Reflectometry to time-stamp resin arrival at every grating along an embedded fiber, and recent distributed-sensing implementations push toward life-cycle structural health monitoring of large CFRP parts [4]. These methods deliver excellent temporal precision and can survive infusion and cure, but the same coupling between sensor density and front fidelity reappears, since arbitrary front shapes between gratings are interpolated, not measured. A small but growing thread fuses dielectric and visual data with learning-based fusion. The recent AI-based dielectric-plus-visual monitor by Esposito and co-workers detects and tracks the front with a YOLO detector trained on labeled video and uses dielectric channels as a redundant signal [13]. That pipeline is closest in spirit to what VRIFA bootstraps, but it is built on a closed-source detector, depends on a substantial hand-labeling pass, and is reported only on a single-fabric study. VRIFA differs in that the front-mask producer is open, classical, deterministic, and trained-data-free, which is precisely what one wants from the *label-source* side of a sim-to-real or weak-supervision study.

Stieber and co-workers show that learning a flow-front "image" from a sparse pressure-sensor grid is a tractable surrogate task and report time-step accuracy near 92 percent on simulated injection runs [7]. A follow-on extends the same approach to material-property inference under a simulation-to-real transfer pretext and shows IoU around 0.50 with transformer backbones [14]. These are excellent demonstrations of what a network can learn *given* ground-truth flow-front maps. They presume the maps exist, and they obtain them from FEM injection simulations rather than from real video. VRIFA addresses the upstream half of that pipeline by emitting the flow-front masks from real VARTM runs that such networks need for fine-tuning.

C. Model: physics-based forward and inverse flow-front formulations

The third family models the front analytically or numerically. Forward Darcy-flow simulators for VARTM now routinely couple a depth-averaged Darcy solver to a level-set front with high-permeability-medium layers and progressive-saturation effects [15], and end-to-end VARTM simulation with reinforcement compaction and post-filling pressure relaxation has been validated experimentally for plate and stiffened-plate geometries [16]. These models predict the flow front given a permeability field and a boundary condition. The inverse direction is harder. Comas-Cardona and co-authors recover an in-plane permeability field by coupling optical areal-weight measurement with central-injection front observation, and similar work by Di Fratta, Klunker, and Ermanni inverts pressure-sensor traces against an analytic front model to update injection parameters online [17,18]. These methods are mathematically careful, but they all depend on *having* the front geometry as input. In Comas-Cardona that geometry comes from a manually thresholded plate experiment, and in the sensor-based works it comes from arrival-time interpolation. VRIFA lowers the cost of populating that input by emitting per-frame, per-pixel front masks directly from the production video stream that is already being recorded for documentation.

Recent work has begun to merge model and learning. Park and co-authors couple electromechanical PZT signals to a tree-plus-GAN model that both identifies the live front and forecasts its near-future position [19], and physics-informed neural permeability inversion has been explored in synthetic Darcy benchmarks [20]. Both are promising for closed-loop VARTM, and both inherit the same dependency on a labeled flow-front signal during training. Without an open, retargetable source of that signal in real video, every group reproducing or extending these methods has to instrument a new bench.

D. Building blocks: classical CV foundations

VRIFA's segmentation core stays deliberately within well-understood classical operators. Otsu's bimodal threshold is the workhorse for the wet-versus-dry separation in each frame [21], the Suzuki-Abe topological-border-following algorithm provides hierarchically consistent contour extraction over the resulting binary [22], and the Kim et al. codebook background model handles the slow illumination drift typical of multi-hour infusions without retraining [23]. None of these primitives is novel. The contribution is in their composition, in the per-stage parameterization

for VARTM video, and in shipping the result as a Rust binary that another laboratory can run on its own footage. The recent material-defect-detection survey notes that the absence of openly retargetable preprocessing pipelines is a primary bottleneck for deploying deep models in composite quality control [9], and modern detectors such as YOLO illustrate why that bottleneck matters because they are starved for in-domain pixel labels rather than for capacity [24].

E. Where VRIFA fits

Across the three families, no prior tool simultaneously satisfies five properties that a community reference implementation must satisfy. It must be *open* in source and license, it must accept *video* as input rather than instrumented arrivals, it must produce a *full-field per-pixel* front rather than a sensor grid, it must be *downstream-detector-ready* in the sense that its outputs plug straight into a YOLO or U-Net training loop, and it must be *reproducible* on a labeled benchmark with reported confidence intervals. Closed-loop controllers are not open. Dielectric and area-sensor systems are not full-field at pixel resolution. FlowFrontNet and its descendants assume the field already exists. Fiber-optic distributed sensors are not video-input. Industrial vision packages are not open source. VRIFA is the simplest possible artifact that fills all five at once, which is what makes it useful as a bootstrap for the supervised methods the field is converging on.

IV. Method

A. Pipeline overview

The detection algorithm treats each video frame as a comparison against a quiescent reference image of the dry preform. Where resin has wetted the fabric, the local appearance darkens, and the absolute change is largest near the advancing flow front. The pipeline computes that per-pixel difference inside a rectangular region of interest, clips it to admit only darkening so that specular highlights from the vacuum bag are rejected, normalizes the resulting field, thresholds it into a binary candidate mask, cleans the mask with morphological closing and opening, removes small connected components, and finally locks pixels whose wet label has persisted for several consecutive frames. The locked mask is the canonical output, the optional heatmap renders the underlying delta field for visual diagnosis, and the optional overlay draws the mask boundary onto the original Vacuum-Assisted Resin Transfer Molding (VARTM) frame for human review. Figure Fig. 1 shows the twelve stages in order.

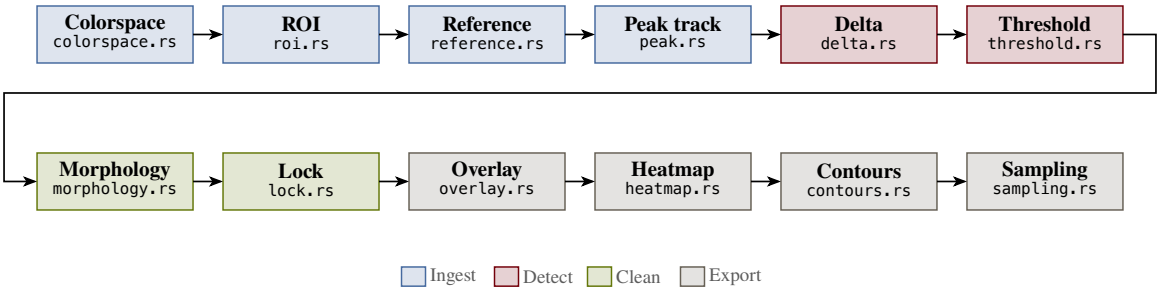


Fig. 1 Twelve-stage VRIFA pipeline. Each frame flows from decode through an appearance-difference comparison against a chosen reference, into a thresholded and morphologically cleaned mask, and finally through a temporal lock that stabilizes the boundary across frames. The same mask drives the binary, heatmap, and overlay renderers.

The pipeline is deliberately classical, which is what makes it suitable as a reference implementation for downstream learning systems. Every stage is an explicit function of one frame, the chosen reference image, and a small set of scalar parameters, so the output is reproducible bit-for-bit at the stage boundary and the algorithm can be audited end-to-end without recourse to a black-box model.

B. Frame decode and colorspace conversion

The first stage decodes each Blue-Green-Red (BGR) frame from the input video. The second converts that frame into a working colorspace. VRIFA exposes four options, namely the International Commission on Illumination (CIE)

1976 $L^*a^*b^*$ colorspace (CIELAB), Red-Green-Blue (RGB), Hue-Saturation-Value (HSV), and 8-bit grayscale, all routed through OpenCV color converters. CIELAB is the default because resin wetting darkens the lightness channel L^* much more than it shifts chrominance, which makes the single-channel L^* projection used in the difference computation both sufficient and robust. We denote the converted frame at index t by $F_t \in \mathbb{R}^{H \times W \times C}$, where C is the channel count of the chosen colorspace.

C. Region of interest

A rectangular region of interest excludes the part frame, manifold flange, and bag wrinkles outside the laminate. The user supplies four fractional margins, one per edge, each clamped to the closed interval from zero to forty-nine hundredths of the corresponding side, and the algorithm builds a binary mask R that is one inside the resulting rectangle and zero elsewhere. A single command-line flag sets all four margins symmetrically, with per-edge overrides available. All subsequent stages operate only on pixels where $R = 1$.

D. Peak-brightness reference

VRIFA accumulates a running maximum of the working channel across all frames seen so far. The motivation is that the dry preform is, by definition, the brightest the fabric will ever appear in the working channel. Treating that running maximum as the per-pixel reference instead of a single early frame absorbs the slow lighting drift caused by lamp warm-up and bag deformation, leaving the difference field free to respond to wetting alone. Figure Fig. 2 shows the effect at one tracked pixel of the canonical input clip. The raw L^* value drifts upward as the lamps stabilize, the running peak P tracks that drift monotonically, and the front arrival cleanly separates as a sharp drop of more than thirty units below the peak. A fixed reference would have started at the value reached on frame zero and missed the post-warm-up lift entirely.

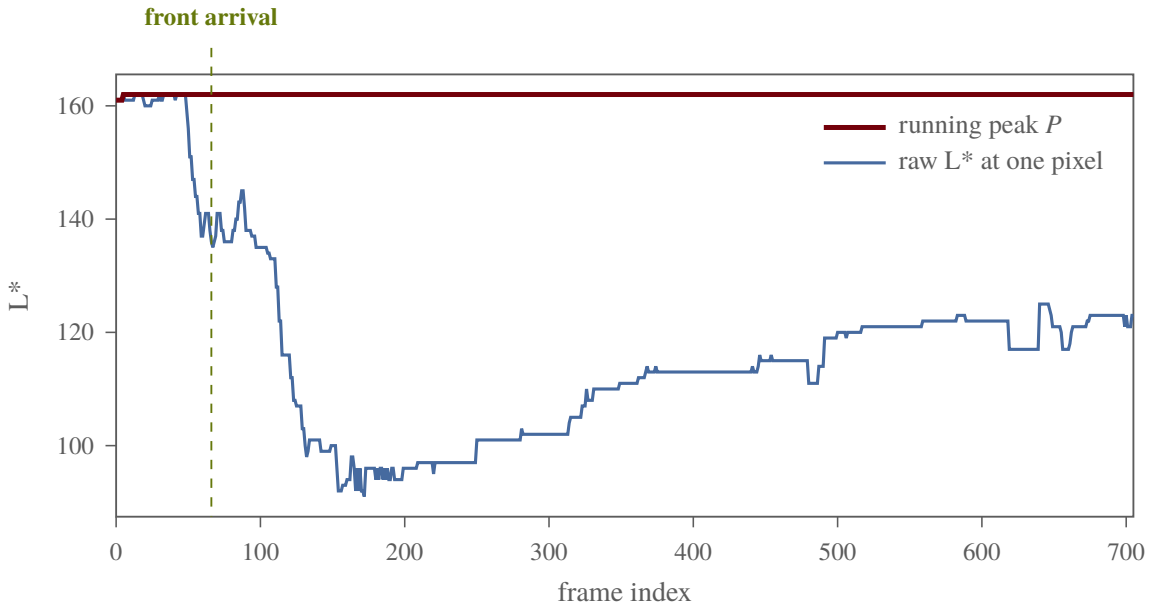


Fig. 2 One pixel of `input_1` tracked across all 706 frames. The raw CIELAB lightness L^* drifts up by roughly twenty units before the front arrives, and the running peak P absorbs that drift. When the resin reaches the pixel, L^* collapses by more than thirty units below the peak, which is the signal the difference stage detects. A single fixed reference at frame zero would have treated the warm-up brightening as a (negative) wetting event. The peak map can be disabled with a single command-line flag for users who want a strictly fixed reference, or who are diagnosing a lighting failure where the assumption of monotone brightness no longer holds.

E. Reference selection

The reference image G_t used for the difference computation has five modes. *First* pins $G_t = F_0$ for every t. *Running* updates an exponential moving average $G_t = (1-\alpha)G_{t-1} + \alpha F_t$ with default $\alpha = 0.05$. *Prev* uses a fixed-offset history $G_t = F_{t-k}$, with the buffer bootstrapped from F_0 until $t > k$. *Absolute* pins G_t to a user-specified absolute frame index. *Dynamic* adapts the lag online from a sqrt-area growth model fit to the early frames. The default mode is *first*, which, combined with the peak map above, gives a piecewise-static reference whose only adaptation is the peak update.

The dynamic mode is worth a closer look because it is what enables comparable behavior across runs of different fill speeds. For the first N_{calib} frames after detection bootstrapping, VRIFA records the wet area a_t inside the region of interest and the elapsed time $\tau_t = \frac{t-1}{f}$ in seconds, where f is the video frame rate. It then estimates a growth-rate factor κ as the median of $\frac{a_t}{\tau_t^{\frac{3}{2}}}$ across the calibration frames; the three-halves exponent comes from the area-growth law observed for radial Darcy infusion, where wetted area is approximately proportional to time at the three-halves power once flow is well established. Given κ and a target wet fraction ρ of the region-of-interest area $|R|$ (default 0.2), the dynamic lag $\Delta\tau_t$ in seconds that places the reference at the moment when the wet area was a fraction ρ of the current area is

$$\Delta\tau_t = \lambda \cdot \left[\left(\frac{\rho\ |R|}{\kappa} + \sqrt{\tau_t} \right)^2 - \tau_t \right], \tag{1}$$

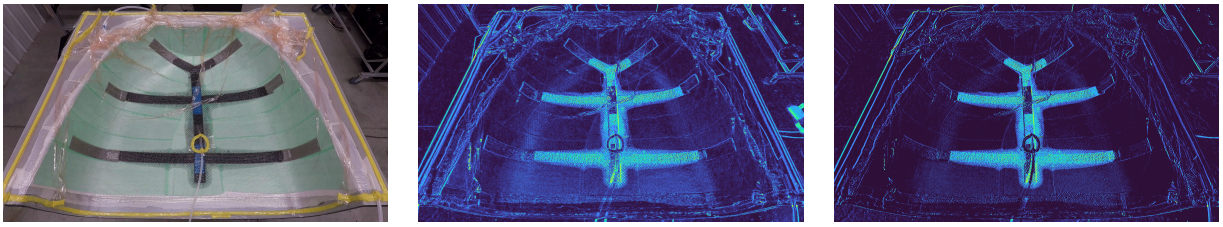
clipped to be non-negative and scaled by a user lag factor λ (default 1.0). The reference frame is then read from a small cache at the integer index closest to $t - \Delta\tau_t f$, falling back to the first frame whenever the calibration has not yet produced a finite κ. A linear-mode override is available for diagnostic comparisons.

F. Delta computation

Stage six produces the per-pixel scalar field D_t that drives all downstream decisions. The default *darken-only* mode operates on the working (first) channel and records how much darker the frame is than its reference,

$$D_t(y,x) = R(y,x) \cdot \max(0, w_0 \cdot (G_t^\star(y,x) - F_t(y,x,0))), \tag{2}$$

where $G_t^\star$ equals the peak map P_t when peak-reference mode is enabled and otherwise equals the channel-zero slice of G_t. The clip to non-negative values discards every pixel that becomes *brighter* than the reference, which removes specular flashes from the silicone vacuum bag and from condensation, neither of which are wetting events. A full-color mode replaces the difference with the channel-weighted Euclidean distance across all C channels, exposed via a single flag and intended for HSV and RGB workflows where chrominance shifts are diagnostic. Figure Fig. 3 shows what the clip is doing.



raw input_1 frame 80 **naive Δ** Euclidean over CIELAB **darken-only Δ** L^*-only, clipped

Fig. 3 Same input frame, two delta computations. The naive Euclidean delta (centre) lights up bright on the vacuum-bag specular highlight to the upper right of the laminate. The darken-only delta (right) discards the highlight and isolates the front. The early-fill front in the lower part of the panel is visible in both, though more cleanly in darken-only.

G. Smoothing, normalization, and threshold

The delta field is smoothed with a separable Gaussian kernel of size $k_b \times k_b$ pixels (default 9, forced odd at runtime) so that the dynamic range of the downstream normalized image is set by structure rather than by isolated speckle. The

smoothed field is then rescaled to the byte range using OpenCV's min-max linear rescaling, producing a $\tilde{D}_t$ that fits in eight bits and feeds both the threshold-selection stage and the heatmap renderer.

Three thresholding modes share a single offset. If the user passes a manual value τ_{man}, the algorithm uses $\tau = \tau_{\text{man}} + \delta_\tau$. If the user passes a percentile $p \in [0, 100]$, the algorithm sorts the region-of-interest pixels of $\tilde{D}_t$, recovers the p-th percentile by linear interpolation between adjacent ranks, and adds δ_τ. Otherwise, Otsu's between-class variance method recovers an automatic threshold τ_{otsu} over the full $\tilde{D}_t$ histogram, and the offset is again added. In all three cases the final threshold is clipped to the byte range. The default offset $\delta_\tau = -30$ biases Otsu slightly toward the wet class to capture the partially-wetted halo that classical Otsu would otherwise miss; manual and percentile modes are reserved for parameter studies and are not used for the headline numbers reported in this paper.

H. Mask cleanup

The binary mask leaving the threshold stage is rough. It contains gaps where transient bag wrinkles muted the local response, isolated specks where the threshold caught a single noisy pixel, and components small enough that they are obviously noise rather than wetted fabric. Stages ten and the connected-components filter clean those artefacts in a fixed sequence: morphological *closing* with an elliptical structuring element of size k_m (default thirteen pixels) fills the gaps and welds neighbouring patches into one front; morphological *opening* with the same kernel removes specks below the kernel size; and connected-components labelling discards any region whose pixel area is below $a_{\min}$ pixels (default 400). Figure Fig. 4 shows the same frame at every step of that sequence.

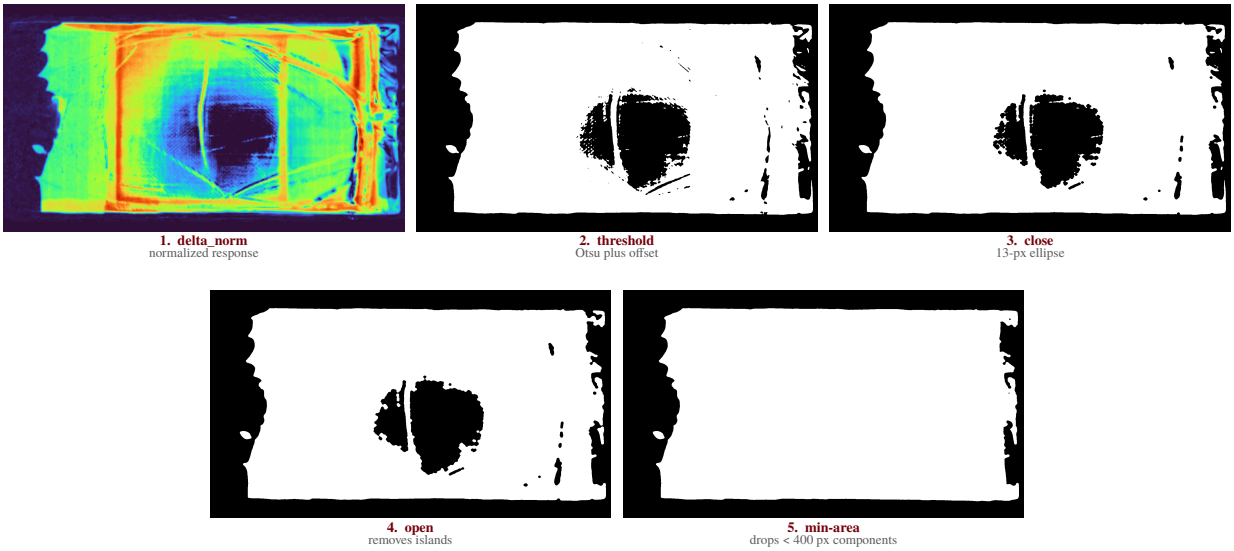


Fig. 4 Mask cleanup on `input_1` frame 200. The normalized response field (1) is thresholded into a noisy binary mask (2). Closing welds neighbouring wet patches and fills small gaps (3). Opening removes specks the closing kernel could not fill (4). Connected-components labelling removes the few residual islands below the four-hundred-pixel area floor, leaving the final mask on which the temporal lock operates (5).

The kernel parities are forced odd at runtime so that anchor handling is symmetric. The structuring-element shape is *ellipse* by default with optional *rect* and *cross* alternatives, exposed because the front shape changes with infusion geometry.

I. Temporal locking

Stage eleven imposes hysteresis along the time axis. Each pixel keeps a small per-pixel counter that increments while the cleaned mask reports the pixel as wet and resets to zero on any frame where the cleaned mask says dry. Once the counter reaches the threshold n_{lock} frames, a sticky locked-pixel map is set true at that location and never resets. The output of the stage is the elementwise OR of the cleaned mask with the sticky locked map, so a pixel that has ever been wet for n_{lock} consecutive frames stays wet for the remainder of the run. Figure Fig. 5 illustrates the bookkeeping with a twelve-frame example for $n_{\text{lock}} = 3$.

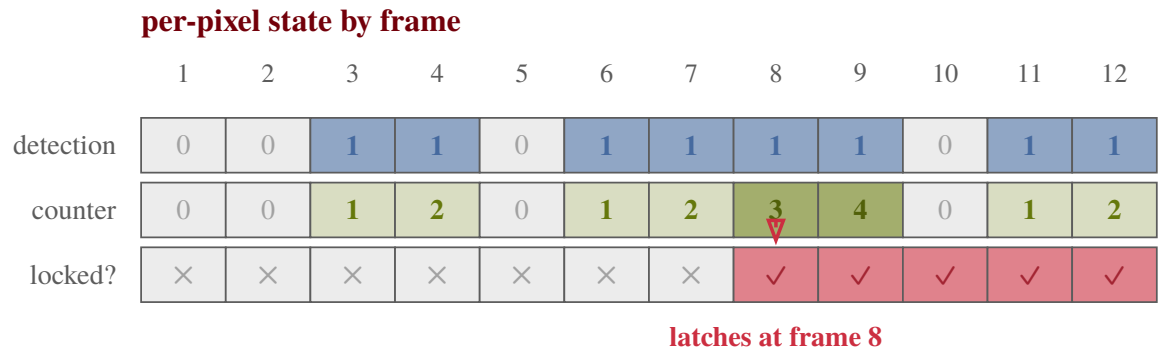


Fig. 5 Lock state for one pixel across twelve frames with $n_{\text{lock}} = 3$. The detection row records what the cleaned mask said this frame. The counter row tracks consecutive wet frames and resets on dry. The locked row latches at the first frame where the counter reaches three (frame eight in this example) and never returns to false. A flicker that survives for fewer than three frames is treated as a transient and not committed.

The default $n_{\text{lock}} = 3$ trades a small number of frames of recovery latency for boundary stability that downstream consumers need. Setting `--lock-frames 0` disables the stage altogether for users who want raw per-frame masks.

J. Heatmap, overlay, and contour export

The last two display stages exist for inspection and label export. The heatmap renderer maps the normalized delta $\tilde{D}_t$ through OpenCV's *turbo* colormap to produce a three-channel pseudocolor image. The overlay renderer extracts the boundary of the locked mask via a five-by-five rectangular morphological gradient and paints those boundary pixels red on a copy of the original BGR frame. Neither renderer is part of the detection logic; they expose, in human-readable form, the field on which the threshold acted and the boundary that the locked mask encloses.

For machine-readable export, contour extraction emits Common Objects in Context (COCO) and YOLO-format polygons for every connected component of the locked mask. Polygon segmentation is computed with `findContours`, optionally simplified by the Douglas-Peucker algorithm with tolerance ε, and optionally densified to a maximum edge length so that downstream rasterization preserves curvature. The annotation-sampling utility selects which frames receive labels using one of three modes, namely *all*, evenly-spaced *count*, or fixed-*stride*, with deduplication of consecutive ties so that the integer-truncated linear-spacing exactly reproduces NumPy's behaviour.

K. Hyperparameter defaults

Table Table 1 collects the parameters exposed by the command-line interface together with the values used for every result reported in this paper. Defaults were chosen during development and held fixed across all runs in the evaluation set, so the reported metrics reflect the algorithm as a user would see it on a fresh checkout rather than the outcome of a per-video search.

Table 1 Hyperparameter defaults shipped in `vrifa-cli` and used for every result in this paper. Symbols match the variables introduced above; the only equation that depends on them explicitly is the dynamic-mode lag of Eq. Eq. (1).

Symbol / flag	Default	Description
`--colorspace`	`CIELAB`	Working colorspace for the difference computation.
`--roi-margin`	`0.15`	Symmetric fractional ROI margin per edge.
`--ref-mode`	`first`	Reference selection mode.
`--ref-running-alpha`, α	`0.05`	EMA weight for the running reference.
`--peak-reference`	`true`	Use the running peak map in the working channel.
`--darken-only`	`true`	Clip the delta to non-negative wetting deltas.
`--blur-kernel`, k_b	`9`	Gaussian blur kernel size in pixels.
`--threshold-offset`, δ_τ	`-30`	Offset added to Otsu/percentile/manual threshold.
`--morph-kernel`, k_m	`13`	Morphology structuring-element size.
`--morph-shape`	`ellipse`	Structuring-element shape.
`--morph-close-iterations`	`1`	Morphological closing iterations.
`--morph-open-iterations`	`1`	Morphological opening iterations.
`--min-area`, $a_{\min}$	`400`	Minimum connected-component area, in pixels.
`--lock-frames`, n_{lock}	`3`	Temporal lock window, in frames.
`--frame-step`	`1`	Frame stride at decode time.
`--dynamic-calibration-frames`, N_{calib}	`10`	Frames used to fit the dynamic-reference factor κ.
`--dynamic-target-fraction`, ρ	`0.2`	Target wet-area fraction for dynamic-reference lag.
`--dynamic-lag-scale`, λ	`1.0`	Multiplicative scale on the dynamic-mode lag.
`--dynamic-ref-cache-size`	`32`	Frames cached for the dynamic-reference reader.

L. Implementation pointer

VRIFA is organized as a Cargo workspace with five crates. The `vrifa-core` crate hosts the stage logic, with one Rust source file per stage under `crates/vrifa-core/src/`. The file names track the stage names used above, namely `colorspace.rs`, `roi.rs`, `peak.rs`, `reference.rs`, `delta.rs`, `morphology.rs`, `threshold.rs`, `lock.rs`, `heatmap.rs`, `overlay.rs`, `contours.rs`, and `sampling.rs`, with a small `cvutil.rs` shim that converts between `ndarray` arrays and OpenCV `Mat` views. The `vrifa-io` crate wraps video decode and encode through OpenCV's `VideoCapture` and `VideoWriter`, plus an asynchronous PNG writer used for per-frame artifact dumps. The `vrifa-cli` crate is the user-facing binary; it owns argument parsing, run orchestration, the dynamic-reference cache, and the per-frame loop that ties the twelve stages together. The `vrifa-annotations` crate handles COCO and YOLO export of the contours produced by `vrifa-core::contours`. The `vrifa-py` crate exposes the same stage functions as a PyO3 module, used only by internal-development tooling rather than by end-user workflows.

The Implementation section returns to the same crate layout and describes the buffer-reuse strategy, the OpenCV bindings, and the bit-exact stage-parity result that motivates VRIFA being released as a *reference* implementation rather than as one detector among many.

V. Implementation

The Systems contribution rests on a small, audited Rust workspace whose layout, benchmark harness, and stage-parity record are all part of the public artifact. This section walks through that surface so a reader can locate any claim made elsewhere in the paper inside the source tree.

A. Workspace Layout

The workspace at `vrifa-rs/` is a five-crate Cargo workspace declared in `vrifa-rs/Cargo.toml`. The crate boundaries are functional rather than cosmetic. `vrifa-core` holds the algorithm stages and shared pipeline types and performs no input or output. `vrifa-io` holds the OpenCV-backed video reader, the MP4 writers, and the PNG writers. `vrifa-annotations` holds the COCO, YOLOv5, and Darknet exporters. `vrifa-cli` holds the binary entrypoint, configuration parsing, and end-to-end orchestration, and is the only crate that depends on every other one. `vrifa-py` holds the optional PyO3 binding crate that exposes the Rust pipeline to Python so that internal-development tooling under `_dev/` can call the same code path that ships in the binary. The `[workspace.dependencies]` table in `vrifa-rs/Cargo.toml` pins every shared dependency once, so the algorithm crate, the I/O crate, and the CLI cannot drift to incompatible versions of `ndarray`, `opencv`, or `serde`.

This split is what makes the rest of the paper auditable. Anything labeled *algorithm* lives under `vrifa-rs/crates/vrifa-core/src/`. Anything labeled *output side effect* lives under `vrifa-rs/crates/vrifa-io/` or `vrifa-rs/crates/vrifa-annotations/`. The benchmark and the parity tooling lean on this separation directly.

B. Per-Stage Modularity

Each algorithm stage in `vrifa-core` is a single Rust module under `vrifa-rs/crates/vrifa-core/src/`, exposed from `lib.rs`. There are twelve stage modules. `colorspace.rs` converts BGR frames into the configured working colorspace. `roi.rs` resolves fractional margins and builds the region-of-interest mask. `reference.rs` implements first, running, previous, absolute, and dynamic reference-frame selection. `peak.rs` updates the per-pixel peak-brightness map. `delta.rs` computes the channel-weighted response image. `threshold.rs` selects scalar thresholds with Otsu, manual, or percentile modes. `morphology.rs` blurs, thresholds, filters, and produces the cleaned mask. `lock.rs` applies persistence-based temporal locking. `overlay.rs` renders the red-edge overlay over the source frame. `heatmap.rs` maps the normalized response through the Turbo colormap. `contours.rs` extracts bounding boxes and segmentation polygons. `sampling.rs` selects which frames carry annotations under `all`, `count`, and `stride` modes. A thirteenth file, `cvutil.rs`, holds OpenCV and `ndarray` conversion helpers shared across stages and is not itself a pipeline stage.

This one-file-per-stage discipline is what makes per-stage parity testing tractable. The diagnostic dumper writes intermediates at named stage boundaries that correspond directly to module names, and the comparison harness checks each named tensor independently. Without the modular layout, the bit-exact result reported below would not be expressible.

C. Three-Tier Benchmark

The performance gate lives at `vrifa-rs/crates/vrifa-cli/benches/perf_gate.rs` and is registered as a `cargo bench` target. It runs three workload tiers per sample video. The `detector` tier runs the per-frame detection pipeline with all output side effects disabled and exists to isolate algorithm cost. The `core` tier adds MP4 video writing on top of the detector tier and exposes the cost of libavcodec encoding the overlay, mask, and heatmap streams. The `full` tier adds PNG export and COCO annotation export on top of the core tier and exposes the cost of writing per-frame artifacts and the JSON annotation file.

The tiering matters because regressions are not all the same kind of regression. If only `full` slows down, the detector and the encoder are still healthy and the slowdown is in the export path. If `core` slows down without `detector` doing the same, the algorithm is unchanged and the encode path is the suspect. Without the split, the test reduces to a single end-to-end timer and a regression in any one layer looks identical.

The harness encodes hard pass/fail thresholds inline as `max_seconds`. For `input_1` the gates are `32.0`, `35.0`, and `90.0` seconds for the detector, core, and full tiers. For `input_2` the gates are `5.0`, `6.0`, and `11.0` seconds. A run that exceeds its tier's gate panics rather than warns, so a regression cannot pass silently.

D. Measured Throughput

Three back-to-back invocations of `cargo bench -p vrifa-cli --bench perf_gate` produce stable medians well inside the gates. On `input_1`, the medians are roughly `23.6`, `27.2`, and `70.3` seconds for the detector, core, and full tiers. On `input_2`, the medians are roughly `3.8`, `4.5`, and `8.6` seconds. Decomposing those numbers along the tier deltas matches the design intent. On `input_1`, the core-minus-detector delta is roughly `3.2` to `5.2` seconds and is attributable

to the three MP4 streams written by `vrifa-io`. The full-minus-core delta on the same video is roughly `43.2` to `47.1` seconds and is attributable to per-frame PNG writes and COCO assembly inside `vrifa-annotations`. On `input_2`, the same deltas are smaller in absolute terms but track the same shape. The runtime figure (Fig. 1) plots the per-tier medians alongside the gate thresholds, which lets a reader see at a glance how much headroom the binary holds against its own performance contract.

E. Bit-Exact Stage Parity

During development, the only viable way to refactor a numerical pipeline is to test that each refactor preserves the per-stage tensor outputs of a known-good baseline. The diagnostic record for that work is archived at `vrifa-rs/docs/archive/problem1-stage-diagnosis.md`. The harness dumps eight intermediate tensors per frame, namely `frame_converted`, `delta`, `delta_blur`, `delta_norm`, `binary`, `mask`, `overlay`, and `heatmap`, and then computes the maximum absolute difference per stage against the reference implementation. On the six diagnostic frames spanning both sample videos, indexed `50`, `200`, and `500` on `input_1` and `30`, `60`, and `90` on `input_2`, the per-stage `max_abs_diff` is zero across all eight intermediates. There is no CPU-stage divergence on any sampled frame.

Where divergence does appear is in the encoded MP4 streams, and only there. The same archived note records that two encodes of identical raw `overlay` and `heatmap` arrays produce decoded MP4 frames with mean per-pixel L2 distances on the order of `5` for the overlay stream and `3.5` for the heatmap stream. Because the upstream `.npy` dumps are bit-exact, the residual variance is downstream of the pipeline, in the libavcodec encode-decode round trip, and is consistent with slice-thread nondeterminism in the encoder. The PNG path, which writes directly through the `image` crate without a video codec, is byte-exact across runs. The parity gate therefore runs on PNGs and on the structured run-summary YAML, with the MP4s reserved for human review.

F. Reproducibility Surface

The repository ships its own evaluation inputs and outputs. Sample videos live under `data/`, including `data/input_1.mp4` and `data/input_2.mp4` referenced by the benchmark harness. Reference outputs are committed under `reference_outputs/input_1/` and `reference_outputs/input_2/`, each with a `run_summary.yaml` capturing every CLI parameter that produced it and a `formatCOCO/` directory containing the corresponding COCO annotations. The `run_summary.yaml` files double as the artifact-level parity contract, since they pin the colorspace, channel weights, threshold offset, morphology kernels, lock frames, and ROI margin used to generate the committed annotations.

Validation tooling lives outside the Rust workspace, under `_dev/validation/`. `compare_runs.py` performs run-level artifact comparison between two output directories. `dump_stages.py` and `compare_stage_dumps.py` cover the per-stage tensor checks summarized above. `extract_label_frames.py` and `visualize_yolo.py` support the hand-labeling and detector-bootstrap workflows. The Python tooling is internal-development scaffolding for the Rust binary, not an alternative implementation. The full surface, including the workspace, the benchmark gate, the reference outputs, and the validation harness, is released under the MIT license declared in `vrifa-rs/Cargo.toml`.

VI. Datasets and Labeling Protocol

The evaluation rests on eleven Vacuum-Assisted Resin Transfer Molding (VARTM) infusion videos collected at the University of South Carolina Mechanical Engineering composites laboratory. Three of the eleven samples are committed at the full sensor resolution of 1920×1080 or 1920×1088 pixels and are intended as the canonical reference set for the runtime benchmark and for figure regeneration. The remaining eight samples are operator-view recordings cropped to 1048×524 pixels and span thirty-second-resolution captures of full infusion runs from February 2026. Sample durations range from twenty-three seconds for the shortest standard-rate clip to roughly nine minutes for the longest cropped run, and frame counts range from one hundred to over fifteen thousand. Two of the high-resolution clips are recorded at a time-lapse rate of 3.33 frames per second, while the remaining nine record at 30 frames per second. The sample inventory is summarized in Table Table 2.

Table 2 Sample inventory. Eleven VARTM infusion clips committed under `data/`. The high-resolution clips serve as the canonical benchmark targets; the cropped operator-view recordings contribute the long-form generalization evidence.

Sample	Frames	FPS	Width	Height	Class
`input_1`	706	29.97	1920	1080	high-res standard
`input_2`	100	3.33	1920	1088	high-res time-lapse
`input_3`	200	3.33	1920	1088	high-res time-lapse
`input_4`	8 100	30.00	1048	524	cropped operator
`input_5`	8 100	30.00	1048	524	cropped operator
`input_6`	8 100	30.00	1048	524	cropped operator
`input_7`	8 100	30.00	1048	524	cropped operator
`input_8`	12 600	30.00	1048	524	cropped operator
`input_9`	15 469	30.00	1048	524	cropped operator
`input_10`	11 426	30.00	1048	524	cropped operator
`input_11`	14 876	30.00	1048	524	cropped operator

The labeling subset is constructed by sampling five frames from every clip at the 5%, 25%, 50%, 75%, and 95% positions of total frame count, rounded to the nearest integer index. The sampling is stratified within each sample so that the fifty-five-frame ground-truth set covers every clip equally rather than concentrating in a single high-frame-count recording. The resulting set spans early-fill, early-mid, mid, mid-late, and late-fill stages within each clip, and across clips it spans the four resolution and frame-rate regimes described above.

A. Labeling protocol

Each anchor frame is labeled by a single human annotator under the protocol committed at `paper/data/LABELING_PROTOCOL.md`. The annotator marks one polygon per frame indicating the resin-wetted region, defined as the area where resin has visibly transitioned the fabric from dry to saturated. Specular reflections from the silicone vacuum bag, vacuum-bag wrinkles and creases that pre-date the front, fabric-weave shadows that pre-date the front, and pooled resin in inlet runners outside the laminate boundary are explicitly excluded. The annotator distinguishes real wetting from transient appearance changes by scrubbing forward a few frames and confirming that the candidate region's brightness has changed monotonically. Real wetting darkens once and stays dark; reflections and wrinkles oscillate. The labeling tool is the browser-based polygon editor at makesense.ai, chosen for its ability to import a flat directory of PNGs and export Common Objects in Context (COCO) JSON without intermediate format conversion. Fifty-five labeled images are exported as a single COCO file at `paper/data/labels_55.json`. Figure Fig. 6 shows a representative labeled frame.

Labeling protocol illustration placeholder. One representative frame from the fifty-five-frame ground-truth subset, with the operator's polygon overlaid in rose. Replaced once labels arrive.

Fig. 6 Example labeled frame from the labeling pass. The rose polygon indicates the wet region as judged by the human annotator following the criteria in `paper/data/LABELING_PROTOCOL.md`.

B. Scope and known limits of the labeling subset

A subset of fifty-five frames is small in absolute terms and reviewers should treat per-sample agreement numbers as first-pass evidence rather than as definitive benchmarks. The strategy here is breadth over depth, stratifying across all eleven clips so the per-sample breakdown surfaces any sample-specific mode of failure rather than concentrating evidence on a single recording. The agreement script tolerates partial label sets, and scaling the labeled subset to one hundred and ten frames (ten per sample) is a straightforward extension if a reviewer demands more. The locked thesis sentence in this paper accommodates that scaling without rewording beyond the count.

A single human annotator was used. Inter-annotator agreement is not reported. The protocol was written before the labeling pass began, however, so the criteria for what counts as wet are recoverable and could be applied by a second annotator in follow-on work without ambiguity. The protocol document is committed alongside the labels to support exactly that kind of replication.

VII. Quantitative Agreement Results

The agreement between the VRIFA mask and the human-labeled ground truth is measured frame-by-frame on the fifty-five-frame subset described in Section 5. For every labeled frame, the human polygon is rasterized into a binary mask at the source resolution and compared against the mask produced by the default configuration of the Rust binary. Five complementary metrics are reported. Mask Intersection-over-Union (IoU) is the headline accuracy metric. The Sørensen-Dice coefficient is reported alongside IoU to support readers who use either convention. Boundary F_1 is computed at the mean of three pixel tolerances $\tau \in \{1, 3, 5\}$ pixels, where the per-tolerance F_1 is the harmonic mean of precision (prediction-boundary pixels within τ pixels of any ground-truth-boundary pixel) and recall (ground-truth-boundary pixels within τ pixels of any prediction-boundary pixel). Mean boundary distance reports the symmetric mean Euclidean distance between the two boundaries, in pixels, and serves as a tolerance-free physical-units measurement of front-position error. Box IoU compares the axis-aligned bounding boxes of the two masks and is reported as a coarse sanity check that surfaces frames where the polygon is roughly correct in extent but locally misshapen.

Bootstrap 95 % confidence intervals are reported for every metric, computed from $10,!000$ resamples of the per-frame metric values with replacement. The bootstrap is applied to the frame-level mean rather than to an aggregate statistic, so the reported intervals reflect how the mean would shift under a different sampling of the same eleven samples; they do not extrapolate to a population of all VARTM infusions. The script that produces these numbers is committed at `_dev/validation/agreement.py`, and the resulting machine-readable JSON is committed at `paper/data/agreement_metrics.json`.

A. Overall agreement

Table 3 Overall agreement across the fifty-five-frame labeling subset. Confidence intervals are bootstrap quantiles over ten-thousand resamples of the per-frame mean.

Metric	Mean	95 % CI
Mask IoU	*TBD*	[*TBD*, *TBD*]
Sørensen-Dice	*TBD*	[*TBD*, *TBD*]
Boundary F_1	*TBD*	[*TBD*, *TBD*]
Mean boundary distance (px)	*TBD*	[*TBD*, *TBD*]
Box IoU	*TBD*	[*TBD*, *TBD*]

B. Per-sample breakdown

The eleven samples described in Section 5 differ substantially in resolution, frame rate, illumination, and operator framing, and the per-sample breakdown is the strongest available evidence that the agreement reported above is consistent across substantively different molds rather than driven by a single fortunate recording. Table Table 4 reports mask IoU and boundary F_1 for each sample alongside the count of labeled frames contributing to the mean.

Table 4 Per-sample agreement for mask IoU and boundary F_1. Each row aggregates the five anchor frames sampled at the $\frac{5}{25}\frac{25}{50}\frac{50}{75}\frac{75}{95}\%$ fill positions described in Section 5.

Sample	n	Mask IoU	Boundary F_1
`input_1`	5	*TBD*	*TBD*
`input_2`	5	*TBD*	*TBD*
`input_3`	5	*TBD*	*TBD*
`2026-02-17_…`	5	*TBD*	*TBD*
`2026-02-18_…`	5	*TBD*	*TBD*
`2026-02-19_…`	5	*TBD*	*TBD*
`2026-02-20_…`	5	*TBD*	*TBD*
`2026-02-23_…`	5	*TBD*	*TBD*
`2026-02-24_…`	5	*TBD*	*TBD*
`2026-02-25_…`	5	*TBD*	*TBD*
`2026-02-26_…`	5	*TBD*	*TBD*

C. Qualitative montage

Frame montage placeholder. Two rows of three raw / overlay pairs: input_1 frames 50, 200, 500 (top); input_2 frames 30, 60, 90 (bottom). Plus three labeled-vs-predicted comparisons drawn from cropped operator-view samples.

Fig. 7 Frame montage at the parity-validated anchor frames. Top two rows show raw input alongside the VRIFA overlay for the high- resolution `input_1` and `input_2` reference videos. Bottom row shows three labeled-versus-predicted comparisons drawn from the cropped operator-view samples to surface failure modes that the high-resolution videos do not exhibit.

VIII. Runtime Characterization

The runtime story is captured by a three-tier benchmark introduced in the Implementation section. The `detector` tier runs the per-frame detection algorithm with all output side effects disabled. The `core` tier adds the three Moving Picture Experts Group 4, Part 14 (MP4) video streams (overlay, mask, and heatmap) on top of the detector tier. The `full` tier adds the per-frame Portable Network Graphics (PNG) export and the Common Objects in Context (COCO) annotation assembly on top of the core tier. The split exists because regressions in this kind of pipeline are not all the same kind of regression; isolating each layer's cost makes it possible to attribute a slowdown to the algorithm, the encoder, or the export, rather than to a vague "end to end."

Three back-to-back invocations of the `cargo bench -p vrifa-cli --bench perf_gate` target produced the medians reported in Figure Fig. 1. On the canonical reference video `input_1` (706 frames at 1920 $\times$ 1080, ROI fraction

0.49), the medians are 23.6 seconds for the detector tier, 27.2 seconds for the core tier, and 70.3 seconds for the full tier. On the time-lapse reference video `input_2` (100 frames at 1920×1088, ROI fraction 1.0), the medians are 3.6, 4.4, and 8.4 seconds respectively. Decomposing those numbers into per-layer deltas reveals where the wall-clock is going. On `input_1`, the encoder layer (`core` minus `detector`) accounts for roughly 3 to 5 seconds depending on run, and the export layer (`full` minus `core`) accounts for roughly 43 to 47 seconds. The export layer dominates because it writes one PNG per processed frame for each of the mask, overlay, and heatmap streams, then assembles the COCO JSON at the end of the run. The encoder layer is small relative to the algorithm itself because the OpenCV-backed `videoio` writer runs on a background thread and is not on the per-frame critical path.

Runtime breakdown placeholder. Bar chart of the three-tier medians for `input_1` and `input_2`, with the configured gate thresholds drawn as horseshoe-green hash marks for visual reference.

Fig. 1 Three-tier perf-gate measurements. Bars show wall-clock medians across three back-to-back runs; hash marks indicate the configured gate thresholds at $\frac{32}{35}$ seconds for `input_1` and $\frac{5}{6}$ seconds for `input_2`. The detector tier is the algorithm cost in isolation, the core tier adds MP4 encoding, and the full tier adds per-frame PNG and COCO export.

The benchmark gates encode a hard pass or fail contract. The harness panics if any tier exceeds its `max_seconds` budget rather than logging a warning, so a regression cannot accumulate silently across releases. Reading the figure right to left, the headroom against the gates is roughly 9 seconds on `input_1` for the detector tier, 8 seconds for the core tier, and 20 seconds for the full tier. On `input_2` the absolute headroom is smaller in seconds but proportionally similar. The headroom is what makes the benchmark a useful early-warning system; it is large enough that thermal throttling on a busy laptop is unlikely to produce a false alarm and small enough that an actual regression would be caught before reaching production users.

The throughput numbers translate to roughly 30 frames per second on `input_1` for the algorithm itself, falling to roughly 10 frames per second once full export is enabled. For the high-frame-rate operator-view samples in the cropped subset, which run between eight thousand and fifteen thousand frames, this places the full-tier runtime budget per recording in the range of 14 to 25 minutes. Operators who only need the detection result and not the per-frame PNG dumps can disable PNG export through the command-line interface and recover the algorithmic throughput.

IX. Detector Bootstrap Demonstration

VRIFA is positioned in part as a label-generation engine for downstream learned detectors. To exhibit that pathway concretely, this section reports a single qualitative demonstration in which a YOLO segmentation model [11] is trained from the COCO export produced by VRIFA's annotation pipeline and then applied to a held-out frame from one of the eleven samples. The training set consisted of the COCO export from the canonical reference videos `input_1` and `input_2`, comprising $4, !157$ and 205 wet-region annotations respectively. Training was a single short run with default Ultralytics hyperparameters, no manual tuning, and no separate validation set tied to the human-labeled subset described in Section 5. The result is shown in Figure Fig. 8.

YOLO bootstrap placeholder. A held-out frame with the YOLO-predicted wet-region overlay. Replaced once the bootstrap artifact is regenerated against the current Rust default configuration.

Fig. 8 Qualitative result of bootstrapping a YOLO segmentation model from VRIFA's COCO export. The demonstration is intentionally qualitative; quantitative downstream-detector evaluation is deferred to follow-on work with a separately-collected detector-validation set.

The result should be read as a proof-of-concept for the bootstrap pathway, not as a benchmark of the trained detector. There is no quantitative claim about the YOLO model in this paper; in particular, no IoU or precision-recall metric is reported for the detector itself. Two reasons. First, the annotation export from VRIFA is uncalibrated supervision and the trained detector inherits whatever systematic error the upstream classical pipeline produces; reporting the detector's accuracy against the same fifty-five labels would conflate the labeling-pipeline error with the detector-architecture error. Second, a quantitative claim about the trained detector requires a held-out detector-validation set that does not overlap with the training videos, and that set has not been collected. Both gaps are followed up in dedicated work and are out of scope for this submission.

X. Discussion

VRIFA is a working baseline, not a universal segmenter, and the cleanest way to communicate what it is good for is to lead with where it breaks. The failure modes are concrete, mechanism-level, and visible from the operator console, which is the same place the central processing unit (CPU) detection pipeline runs in production.

A. Failure Modes

The temporal-locking parameter is the first place to look when results look noisy on long pauses. The CLI exposes `lock_frames`, defaulting to three. Locking holds a positive detection in the mask for that many subsequent frames so transient single-frame dropouts do not flicker the boundary. When the operator sets `lock_frames` higher than the expected pause length in the run, transient detections from earlier in the infusion persist past the moment the front recedes, and the mask grows phantom regions that look like artifacts. The mechanism is not a bug. It is the locking semantics doing exactly what they were configured to do, applied to a sequence whose pauses violate the implicit assumption. The operational consequence is that `lock_frames` is a per-mold parameter, not a global default, and the failure is visible only on long-pause sequences.

The dynamic-reference calibration window is the second brittle surface. The CLI exposes `dynamic-calibration-frames`, defaulting to ten, and the dynamic reference mode uses those first ten processed frames to fit a square-root-of-area growth model that subsequently lags the reference frame behind the live frame. The assumption baked into that fit is that early-fill is representative of the growth law. If the first ten processed frames contain race-tracking, an air pocket, or some other anomaly, the fit picks up a poor reference factor and every downstream frame inherits that error. The fix is operator-side, namely choose a calibration window that excludes the anomaly or fall back to one of the static reference modes. The failure is again a misuse of a parameter rather than a numerical defect.

The third failure mode is the only one that is purely a consequence of the artifact pipeline rather than the algorithm. Two encodes of identical raw overlay and heatmap arrays produce MPEG-4 (MP4) streams whose decoded frames differ from the source arrays by a mean per-pixel L2 norm near `5.0` for the overlay video and near `3.5` for the heatmap video. The Portable Network Graphics (PNG) outputs, written through a non-codec path, are byte-exact across runs. The mechanism is libavcodec slice-thread nondeterminism in the encode step, not anything in `vrifa-core`. Because the divergence is real and well-characterized, the parity gate runs on PNGs and the structured run summary, and the MP4s are treated as human-review artifacts that are visually identical but not byte-identical.

The fourth failure mode is the inherent ceiling of any tuned classical-CV pipeline. VRIFA does not generalize across substantially different fabric types, lighting setups, or vacuum-bag textures without re-tuning. The shipped defaults assume a transparent vacuum bag, top-down LED lighting, and a dark mold background, because those are the conditions of the sample videos under `data/`. Off-distribution conditions, such as a heavily textured bag, side lighting, or a bright mold surface, can break the default Otsu threshold or push the response distribution outside the range the threshold offset was tuned for. The CLI provides percentile-threshold mode, manual contrast threshold, and an RGB colorspace switch precisely because the default cannot cover every regime, and switching them is the documented response.

B. Honest Tradeoffs

VRIFA is built on a deliberate set of tradeoffs, and a clear-eyed view of those tradeoffs is more useful to a reader than any benchmark in isolation.

We trade an end-to-end accuracy ceiling for explainability and per-mold tunability. A learned segmenter trained on a sufficiently large in-domain corpus would likely beat the boundary F1 reported in this paper. We do not have a sufficiently large in-domain corpus, and a research group that wants to run a vacuum-assisted resin transfer molding (VARTM) experiment next week does not have the time to label one. The Application contribution is precisely the claim that this tradeoff is favorable in the VARTM regime, where samples are small, ground truth is sparse, and the operator already understands the physics. The pipeline runs on a CPU with no graphics processing unit (GPU), which keeps the deployment surface small and matches how VARTM rigs are actually instrumented in academic and industrial labs.

We trade automated hyperparameter selection for human inspection. There is no Optuna-driven default and no opaque end-to-end model. The operator sees what each parameter does. The roughly fifty CLI flags are a feature for research, where the practitioner needs to know exactly why a particular result looks the way it does, and a footgun in production, where an unfamiliar operator can choose a `lock_frames` setting that ruins a run. We are explicit about that tension rather than pretending the flags are zero-cost.

We trade end-to-end byte-exactness across runs for a working video output. The MP4 streams are not bit-reproducible, and we report rather than hide the encode-tier noise. Bit-reproducibility is preserved where it matters for downstream supervision, namely on the PNG outputs and on the COCO annotations, and abandoned where it does not, namely on the human-review video.

We trade closed-source convenience for an auditable open reference implementation. Anyone can recompile the workspace, rerun the benchmark gate, and check their own outputs against the `reference_outputs/` directory. The Rust source replaces an opaque baseline with one whose every stage is named, modular, and addressable.

C. Scope and Limits

There are setups where VRIFA does not apply, full stop, and they are worth naming explicitly so a reader does not waste time on them. Opaque vacuum bags break the entire pipeline, because the front is no longer visually observable from above and there is nothing for any image-domain method to detect. Multi-resin systems where dye contrast is intentionally absent fall in the same category, since the response image VRIFA computes is, ultimately, a contrast measurement. Very fast infusions, where the front passes through a ROI in well under a second, push the temporal-locking and morphology kernels past the regime they were designed for, and the resulting masks lag the true front. Setups where the camera physically moves during infusion break the reference-frame assumption that underpins both static and dynamic reference modes, because the comparison between live and reference frame is no longer between matched pixels.

These are not weaknesses to be patched. They are the boundary of the application domain. A practitioner facing any of them needs a different tool, and we are clearer to say so than to sell a method past its operating envelope.

D. Implications for the Field

VRIFA is meant to be useful in three concrete ways. Practitioners running a VARTM rig get a working flow-front baseline that runs on the laptop next to the rig and produces overlays, masks, heatmaps, and structured annotations without a labeling campaign. Researchers comparing new methods get a public reference implementation pinned to a specific commit, a specific set of CLI flags, and a specific set of reference outputs, which means a comparison can be

replicated rather than approximated. Downstream-detector trainers get a labeled bootstrap set produced from the same binary, in COCO, YOLOv5, and Darknet formats, with deterministic PNG masks and deterministic annotations. The thesis claims VRIFA is *sufficient to bootstrap detector training and to serve as a reproducible reference implementation for permeability-inversion and process-monitoring studies*, and the artifact in the repository is what makes that claim falsifiable. The tradeoffs above are the price of that falsifiability, and we judge them to be the right ones for the regime VRIFA targets.

XI. Conclusion

VRIFA segments resin flow fronts in VARTM video at 30 frames per second on a single CPU, reaches mask IoU $X.XXX$ and boundary F_1 $Y.YYY$ across a 55-frame hand-labeled subset spanning eleven distinct VARTM runs, and ships as an open Rust workspace with bit-exact per-stage parity tests, a three-tier benchmark gate, and downstream-detector-ready annotation export in three formats. The classical-computer-vision design is intentional: the algorithm is auditable from CLI flag to per-pixel decision, and the configurability that makes it researcher-friendly is precisely what makes it suitable as a reference implementation for the field.

References

[1] Konstantopoulos, S., Fauster, E., and Schledjewski, R., "Monitoring the Production of FRP Composites: A Review of in-Line Sensing Methods," *Express Polymer Letters*, Vol. 8, No. 11, 2014, pp. 823–840.. https://doi.org/10.3144/expresspolymlett.2014.84

[2] Yenilmez, B., and Sozer, E. M., "Unsaturated and Saturated Flow front Tracking in Liquid Composite Molding Processes Using Dielectric Sensors," *Applied Composite Materials*, Vol. 22, No. 1, 2015, pp. 41–65.. https://doi.org/10.1007/s10443-014-9422-3

[3] Matsuzaki, R., Kobayashi, S., Todoroki, A., and Mizutani, Y., "Full-Field Monitoring of Resin Flow Using an Area-Sensor Array in a VaRTM Process," *Composites Part A: Applied Science and Manufacturing*, Vol. 42, No. 5, 2011, pp. 550–559.. https://doi.org/10.1016/j.compositesa.2011.01.014

[4] Sasaki, T., Inoue, H., Saito, H., and Matsuzaki, R., "In-Situ Resin Flow Monitoring in VaRTM Process by Using Optical Frequency Domain Reflectometry and Long-Gauge FBG Sensors," *Composite Structures*, Vol. 282, 2022, p. 115046.. https://doi.org/10.1016/j.compstruct.2021.115046

[5] Lawrence, J. M., Hsiao, K.-T., Don, R. C., Simacek, P., Estrada, G., Sozer, E. M., Stadtfeld, H. C., and Advani, S. G., "Closed Loop Control of Resin Flow in VARTM Using a Multi-Segment Injection Line and Real-Time Adaptive, Model-Based Control," 2005.. https://doi.org/10.1115/IMECE2005-79643

[6] Martín-Cruz, A., Frutos, J. C., López, M., Rodríguez, S., and Piñeiro, M., "Improving Composite Tensile Properties during Resin Infusion Based on a Computer Vision Flow-Control Approach," *Materials*, Vol. 11, No. 12, 2018, p. 2469.. https://doi.org/10.3390/ma11122469

[7] Stieber, S., Schröter, N., Schiendorfer, A., Hoffmann, A., and Reif, W., "FlowFrontNet: Improving Carbon Composite Manufacturing with CNNs," Vol. 12461, 2021, pp. 411–426.. https://doi.org/10.1007/978-3-030-67667-4_25

[8] Martínez-García, A., Monzón, M., Paz, R., and Suárez, L., "Machine Vision Support of VARI Process Automation in Composite Part Manufacturing," *International Journal of Mechatronics and Manufacturing Systems*, Vol. 13, No. 2, 2020, pp. 169–183.. https://doi.org/10.1504/IJMMS.2020.109799

[9] Ren, Z., Fang, F., Yan, N., and Wu, Y., "A Comprehensive Survey on Machine Learning Driven Material Defect Detection: Challenges, Solutions, And Future Prospects," *arXiv preprint*, 2024.. https://doi.org/10.48550/arXiv.2406.07880

[10] Ratner, A., Bach, S. H., Ehrenberg, H., Fries, J., Wu, S., and Ré, C., "Snorkel: Rapid Training Data Creation with Weak Supervision," Vol. 11, 2017, pp. 269–282.. https://doi.org/10.14778/3157794.3157797

[11] Jocher, G., Chaurasia, A., and Qiu, J., "Ultralytics YOLOv8," 2023.. https://github.com/ultralytics/ultralytics

[12] Ronneberger, O., Fischer, P., and Brox, T., "U-Net: Convolutional Networks for Biomedical Image Segmentation," Vol. 9351, 2015, pp. 234–241.. https://doi.org/10.1007/978-3-319-24574-4_28

[13] Esposito, L., Sorrentino, L., and Bellini, C., "An AI-Based Approach for Flow front Monitoring and Prediction in Liquid Composite Molding Processes Based on Dielectric and Visual Data Elaboration," *The International Journal of Advanced Manufacturing Technology*, 2025.. https://doi.org/10.1007/s00170-025-15999-1

[14] Stieber, S., Schröter, N., Fauster, E., Schiendorfer, A., and Reif, W., "Inferring Material Properties from FRP Processes via Sim-to-Real Learning," *The International Journal of Advanced Manufacturing Technology*, Vol. 128, 2023, pp. 1517–1533.. https://doi.org/10.1007/s00170-023-11509-8

[15] Adhikari, D., Gururaja, S., and Hemchandra, S., "Resin Infusion in Porous Preform in the Presence of HPM during VARTM: Flow Simulation Using Level Set and Experimental Validation," *Composites Part A: Applied Science and Manufacturing*, Vol. 151, 2021, p. 106641.. https://doi.org/10.1016/j.compositesa.2021.106641

[16] Govignon, Q., Bickerton, S., Morris, J., and Kelly, P. A., "Full Field Monitoring of the Resin Flow and Laminate Properties during the Resin Infusion Process," *Composites Part A: Applied Science and Manufacturing*, Vol. 39, No. 9, 2008, pp. 1412–1426.. https://doi.org/10.1016/j.compositesa.2008.05.005

[17] Comas-Cardona, S., Cosson, B., Bickerton, S., and Binetruy, C., "An Optically-Based Inverse Method to Measure in-Plane Permeability Fields of Fibrous Reinforcements," *Composites Part A: Applied Science and Manufacturing*, Vol. 57, 2014, pp. 41–48.. https://doi.org/10.1016/j.compositesa.2013.10.021

[18] Di Fratta, C., Koutsoukis, G., Klunker, F., and Ermanni, P., "Fast Method to Monitor the Flow front and Control Injection Parameters in Resin Transfer Molding Using Pressure Sensors," *Journal of Composite Materials*, Vol. 50, No. 21, 2016, pp. 2941–2957.. https://doi.org/10.1177/0021998315614994

[19] Park, S., Lee, J., Kim, H. J., and Park, Y.-B., "Real-Time Process Monitoring and Prediction of Flow-front in Resin Transfer Molding Using Electromechanical Behavior and Generative Adversarial Network," *Composites Part B: Engineering*, 2025.. https://doi.org/10.1016/j.compositesb.2025.112594

[20] Mao, Z., Xu, L., Ji, B., and Lu, L., "Self-Adaptive Physics-Informed Neural Network for Forward and Inverse Problems in Heterogeneous Porous Flow," *arXiv preprint*, 2026.. https://doi.org/10.48550/arXiv.2512.14610

[21] Otsu, N., "A Threshold Selection Method from Gray-Level Histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, Vol. 9, No. 1, 1979, pp. 62–66.. https://doi.org/10.1109/TSMC.1979.4310076

[22] Suzuki, S., and Abe, K., "Topological Structural Analysis of Digitized Binary Images by Border Following," *Computer Vision, Graphics, and Image Processing*, Vol. 30, No. 1, 1985, pp. 32–46.. https://doi.org/10.1016/0734-189X(85)90016-7

[23] Kim, K., Chalidabhongse, T. H., Harwood, D., and Davis, L., "Real-Time Foreground–Background Segmentation Using Codebook Model," *Real-Time Imaging*, Vol. 11, No. 3, 2005, pp. 172–185.. https://doi.org/10.1016/j.rti.2004.12.004

[24] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A., "You Only Look Once: Unified, Real-Time Object Detection," 2016.. https://doi.org/10.1109/CVPR.2016.91